

Matteo Palmonari
matteo.palmonari@unimib.it



INSID&S Lab
*Interaction and Semantics for
Innovation with Data & Services*

RDFS:

Basic Reasoning for RDF

RDF: IRIfied Graphs and Triples

<dbr:Electric_Piano, rdfs:label, "Electric Piano"@en>
 <dbr:Electric_Piano, rdfs:label, "Piano Elettrico"@it>
 <dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
 <dbr:Empty_Sky, dbo:artist, dbr:Elton_John>
 <dbr:Sails, dbo:musicalArtist, dbr:Elton_John>
 <dbr:Sails, dbo:album, dbr:Empty_Sky>
 <dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
 <dbr:Elton_John, rdf:type, dbo:Musical_Artist>
 <dbr:Sails, rdf:type, dbo:Single>
 <dbr:Empty_Sky, rdf:type, dbo:Album>
 <dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
 <dbo:Artist, rdfs:subClassOf, dbo:Person>

...

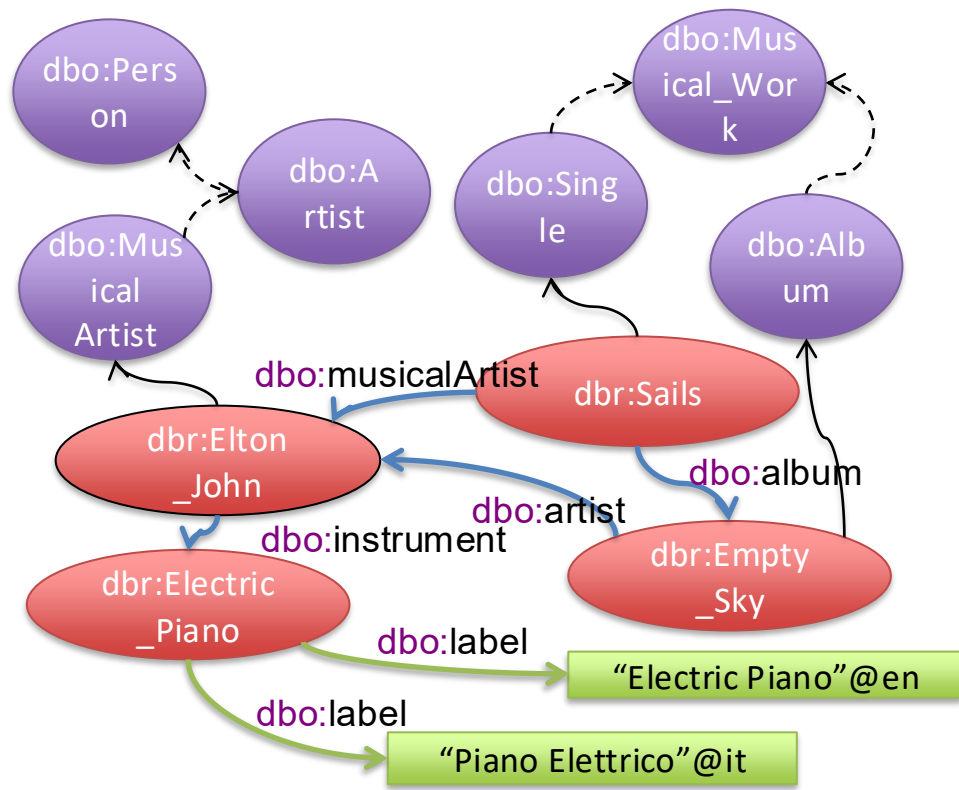
dbr: for <http://dbpedia.org/resource/>

e.g.: http://dbpedia.org/resource/Elton_John

dbo: for <http://dbpedia.org/ontology/>

rdfs for <https://www.w3.org/2000/01/rdf-schema#>

rdf for <http://www.w3.org/1999/02/22-rdf-syntax-ns#>



SPARQL Query: Example

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>
```

```
SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

- Web-based data manipulation language: query, update, etc.
- Inspired by SQL: pattern matching on the data graph
- Graph query patterns: path queries
- Can work without (default) or with reasoning

Modeling Knowledge in KGs

■ Vocabularies as **social agreements**

- Semantic interoperability
- No reasoning

■ Vocabularies as **formal ontologies**

- Semantic interoperability
- Reasoning: new triples + consistency check

Example - Schema.org:Event

schema.org

Custom Search



Home

Schemas

Documentation

Event

[Thing](#) > [Event](#)

An event happening at a certain time and location, such as a concert, lecture, or festival. Ticketing information may be added via the [offers](#) property. Repeated events may be structured as separate Event objects.

[\[more...\]](#)

Property	Expected Type	Description
Properties from Event		
about	Thing	The subject matter of the content. Inverse property: subjectOf .
actor	Person	An actor, e.g. in tv, radio, movie, video games etc., or in an event. Actors can be associated with individual items or with a series, episode, clip. Supersedes actors .
aggregateRating	AggregateRating	The overall rating, based on a collection of reviews or ratings, of the item.
attendee	Organization or Person	A person or organization attending the event. Supersedes attendees .
audience	Audience	An intended audience, i.e. a group for whom something was created. Supersedes serviceAudience .
composer	Organization or Person	The person or organization who wrote a composition, or who is the composer of a work performed at some event.

Modeling Knowledge in KGs

■ Vocabularies as **social agreements**

- Semantic interoperability
- No reasoning

■ Vocabularies as **formal ontologies**

- Semantic interoperability
- Reasoning: new triples + consistency check

(Axiomatic) Ontologies

An **axiomatic ontology** is a formal specification of a conceptualization by means of **a language L** , a set of **logical axioms of L** and a formal semantics that uniquely determines the **meaning** of the terms, i.e., the symbols of L .

To determine the meaning of a term $\rightarrow$ to specify its unambiguous interpretation within a mathematical model



Axiomatic ontologies organize knowledge by defining:

- **Vocabulary:** **symbols** to represent **individuals** (e.g., entities like *Elton John*), **classes** (e.g., entity types like *Musical Artist*), **properties** (e.g., properties to describe entities like *made*)
- **Axioms** to represent the logical constraints that specify their meaning and thus support **automated reasoning (inference)**

Knowledge Graphs & RDF: Recap

■ RDF Knowledge Graphs

- Data model: RDF
- Query language: SPARQL
- Vocabularies and ontologies
- **Basic reasoning: RDFS**
- Advanced reasoning: OWL2

DATA

MODELING

REASONING

KNOWLEDGE
ENGINEERING



RDFS

1 – RDFS & Inference

RDF Schema

■ RDF Schema

- primitives to model classes and properties
- ~ set-based interpretation: by introducing **classes**, subtly introduce also **instances**
- some metamodeling --> skipped in this presentations

rdfs:Resource

All things described by RDF are called **resources**, and are instances of the class **rdfs:Resource**.

_:bob
foaf:Person
xsd:string
"bob"^^xsd:string
rdfs:Class
rdfs:Literal
rdfs:Datatype

rdfs:Literal

The class **rdfs:Literal** is the class of literal values such as **strings** and **integers**.

rdfs:Class

A class refers to a **group of resources**.

All instances of rdfs:Class are known as **instances**. Classes are themselves **resources**.

The **rdfs:subClassOf** property may be used to state that one class is a subclass of another

rdfs:Resource xsd:string foaf:Person

rdfs:Datatype

A Datatype refers to a **group of resources**.

All instances of rdfs:Datatype:

- correspond to the [RDF model of a datatype](#) [>> [RDF12-CONCEPTS](#)]
- are subclasses of rdfs:Literal

"bob"^^xsd:string

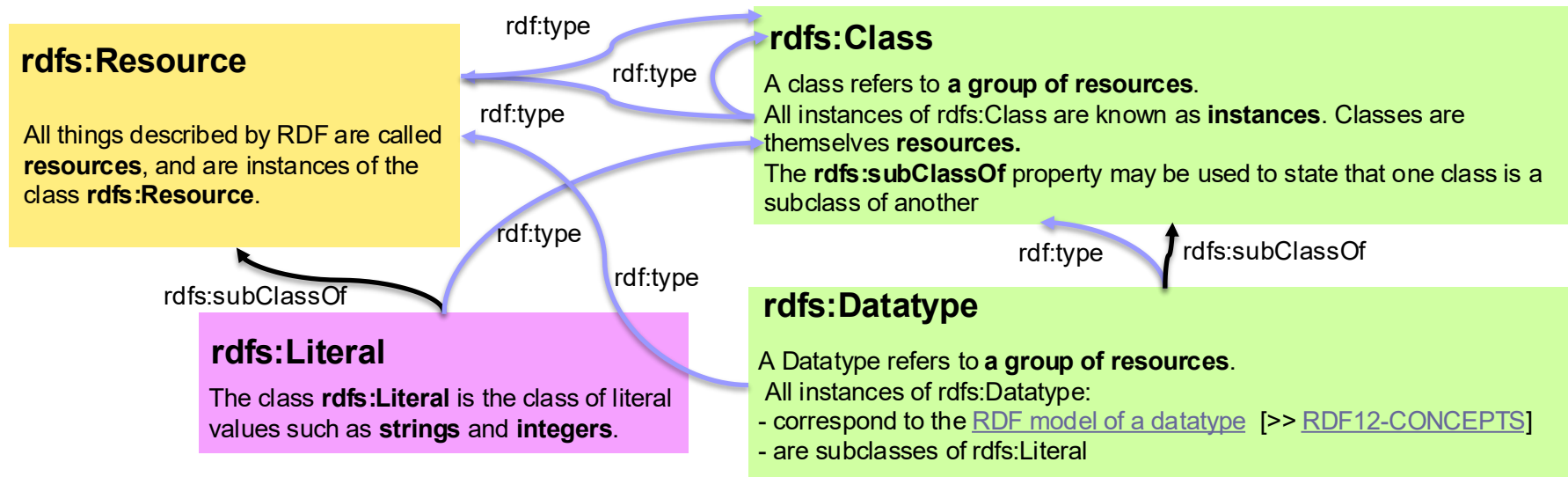
xsd:string

RDF Schema

We skip this!
(metamodeling)

■ RDF Schema

- primitives to model classes and properties
- ~ set-based interpretation: by introducing **classes**, subtly introduce also **instances**
- some metamodeling --> skipped in this presentations



RDFS Primitives for Reasoning

■ Model taxonomies and a bit beyond

- Hierarchical class structures
 - **rdfs:class** and **rdfs:subClassOf**
- Hierarchical property structures
 - **rdfs:subPropertyOf**
- Domain and Range restrictions on properties
 - **rdfs:domain** and **rdfs:range**

■ Basic reasoning tasks

- is-a, membership to classes via class hierarchies and domain/range inference, fact inference via property hierarchies

*RDFS
We present it with its simple
rule-based semantics*

RDFS Rule List

- Contains 13 entailment rules denominated *rdfs*i** for inference over RDFS definitions*:
 - *rdfs:Literal* (*rdfs1*, *rdfs13*)
 - *rdfs:domain* (*rdfs2*), *rdfs:range* (*rdfs3*)
 - *rdfs:Resource* (*rdfs4a*, *rdfs4*, *rdfs8*)
 - *rdfs:subPropertyOf* (*rdfs5*, *rdfs6*, *rdfs7*, *rdfs12*)
 - *rdfs:Class* (*rdfs8*, *rdfs10*)
 - *rdfs:subClassOf* (*rdfs9*, *rdfs10*, *rdfs11*)
 - *rdfs:ContainerMembershipProperty* (*rdfs12*)
 - *rdfs:Datatype* (*rdfs13*)

* Source: <http://www.w3.org/TR/rdf-mt/#RDFSRules>

RDFS

Entailment

Rules (13)

Metamodeling
and RDFS technical
constructs (e.g., lists)

Ex:

- the subject/object of `rdfs:subClassOf` is an `rdfs:Class`
- the object of `rdf:type` is an `rdfs:Class`

Semantics of:

- `subClassOf`
- `subPropertyOf`
- `domain`
- `range`

Rule Name	If E contains:	then add:
rdfs1	<code>uuu aaa lll .</code> where <code>lll</code> is a plain literal (with or without a language tag).	<code>_:nnn rdf:type rdfs:Literal .</code> where <code>_:nnn</code> identifies a blank node allocated to <code>lll</code> by rule <code>rule lg</code> .
rdfs2	<code>aaa rdfs:domain XXX .</code> <code>uuu aaa yyy .</code>	<code>uuu rdf:type XXX .</code>
rdfs3	<code>aaa rdfs:range XXX .</code> <code>uuu aaa vvv .</code>	<code>VVV rdf:type XXX .</code>
rdfs4a	<code>uuu aaa xxx .</code>	<code>UUU rdf:type rdfs:Resource .</code>
rdfs4b	<code>uuu aaa vvv .</code>	<code>VVV rdf:type rdfs:Resource .</code>
rdfs5	<code>UUU rdfs:subPropertyOf VVV .</code> <code>VVV rdfs:subPropertyOf XXX .</code>	<code>UUU rdfs:subPropertyOf XXX .</code>
rdfs6	<code>UUU rdf:type rdf:Property .</code>	<code>UUU rdfs:subPropertyOf UUU .</code>
rdfs7	<code>aaa rdfs:subPropertyOf bbb .</code> <code>uuu aaa yyy .</code>	<code>uuu bbb yyy .</code>
rdfs8	<code>UUU rdf:type rdfs:Class .</code>	<code>UUU rdfs:subClassOf rdfs:Resource .</code>
rdfs9	<code>UUU rdfs:subClassOf XXX .</code> <code>VVV rdf:type UUU .</code>	<code>VVV rdf:type XXX .</code>
rdfs10	<code>UUU rdf:type rdfs:Class .</code>	<code>UUU rdfs:subClassOf UUU .</code>
rdfs11	<code>UUU rdfs:subClassOf VVV .</code> <code>VVV rdfs:subClassOf XXX .</code>	<code>UUU rdfs:subClassOf XXX .</code>
rdfs12	<code>UUU rdf:type rdfs:ContainerMembershipProperty .</code>	<code>UUU rdfs:subPropertyOf rdfs:member .</code>
rdfs13	<code>UUU rdf:type rdfs:Datatype .</code>	<code>UUU rdfs:subClassOf rdfs:Literal .</code>



RDFS semantics (core part of it – most relevant for inferences)

if	then
<code>x rdfs:subClassOf y .</code> <code>a rdf:type x .</code>	<code>a rdf:type y .</code>
<code>x rdfs:subClassOf y .</code> <code>y rdfs:subClassOf z .</code>	<code>x rdfs:subClassOf z .</code>
<code>x p y .</code> <code>p rdfs:subPropertyOf q .</code>	<code>x q y .</code>
<code>p rdfs:subPropertyOf q .</code> <code>q rdfs:subPropertyOf r .</code>	<code>p rdfs:subPropertyOf r .</code>
<code>x p y .</code> <code>p rdfs:domain z .</code>	<code>x rdf:type z .</code>
<code>x p u .</code> <code>p rdfs:range z .</code>	<code>u rdf:type z .</code>

Read out more in RDF Semantics <http://www.w3.org/TR/rdf-mt/>



RDFS semantics (core part of it)

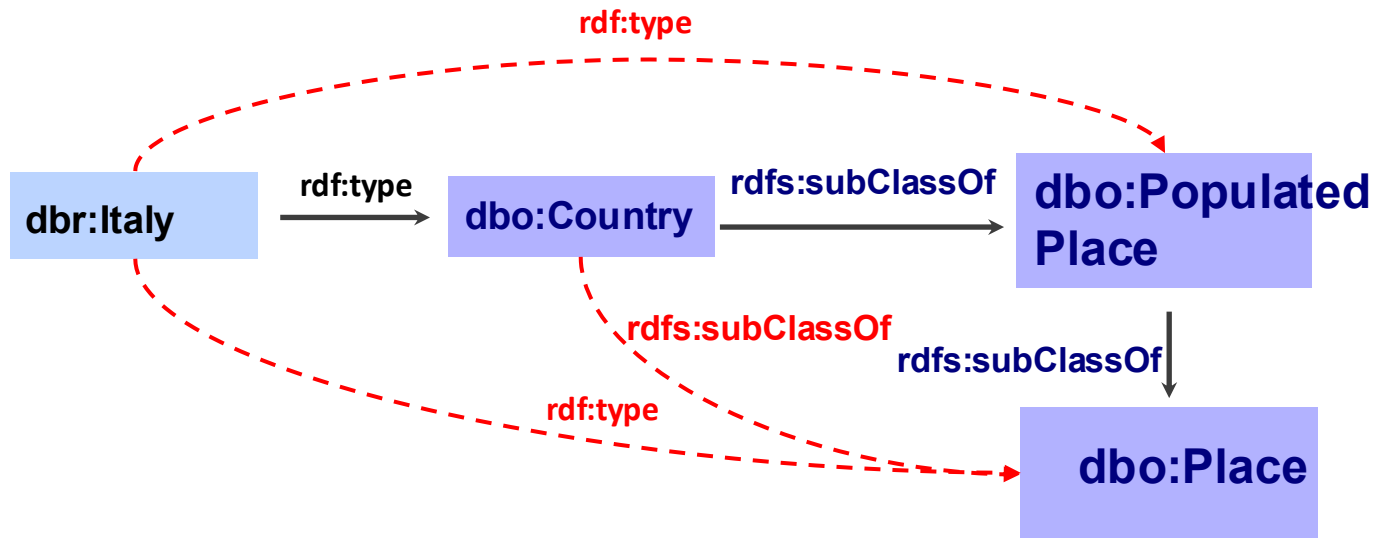


if	then
<code>x rdfs:subClassOf y .</code> <code>a rdf:type x .</code>	<code>a rdf:type y .</code>
<code>x rdfs:subClassOf y .</code> <code>y rdfs:subClassOf z .</code>	<code>x rdfs:subClassOf z .</code>
<code>x p y .</code> <code>p rdfs:subPropertyOf q .</code>	<code>x q y .</code>
<code>p rdfs:subPropertyOf q .</code> <code>q rdfs:subPropertyOf r .</code>	<code>p rdfs:subPropertyOf r .</code>
<code>x p y .</code> <code>p rdfs:domain z .</code>	<code>x rdf:type z .</code>
<code>x p u .</code> <code>p rdfs:range z .</code>	<code>u rdf:type z .</code>

+ rdfs10
(reflexivity)

Read out more in RDF Semantics <http://www.w3.org/TR/rdf-mt/>

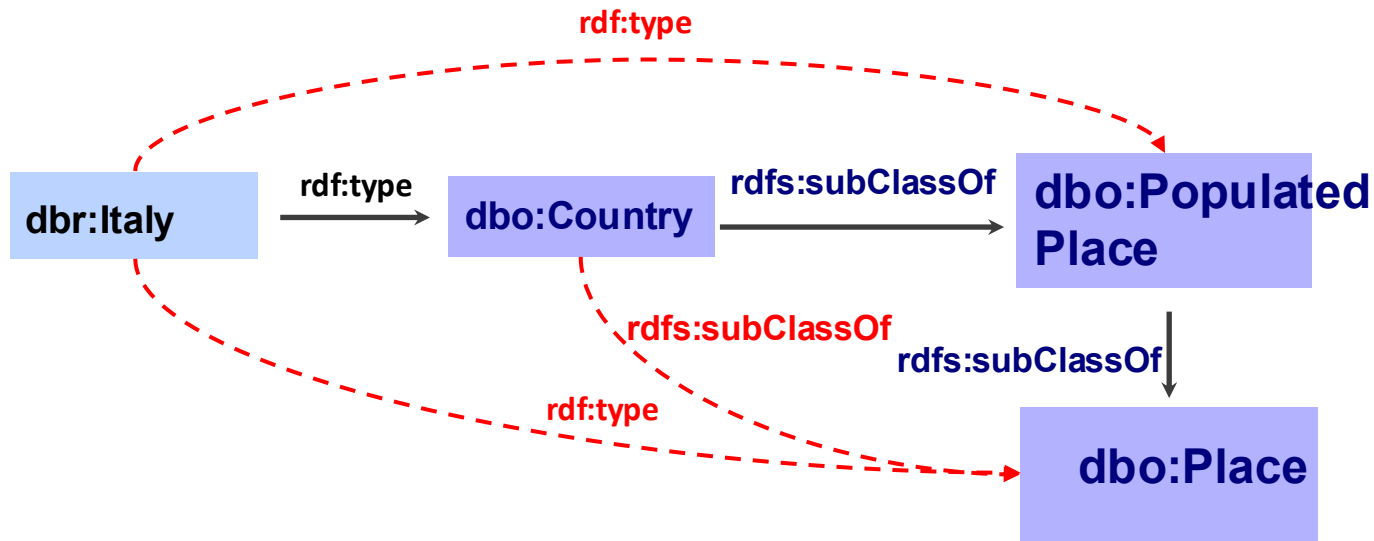
Minimal Reasoning with RDFS



If a Country is a Populated Place **and** a Populated Place is a Place,
then a Country is also a Place

If Italy is a Country **and**
a Country is a Populated Place **and**
a Populated Place is a Place,
then Italy is also a Populated Place and a Place

Minimal Reasoning with RDFS



- *Subclass of* relation is **transitive**, **antisymmetric** and **reflexive**
- An entity is subclass of every superclass of its declared class



RDFS semantics (core part of it)

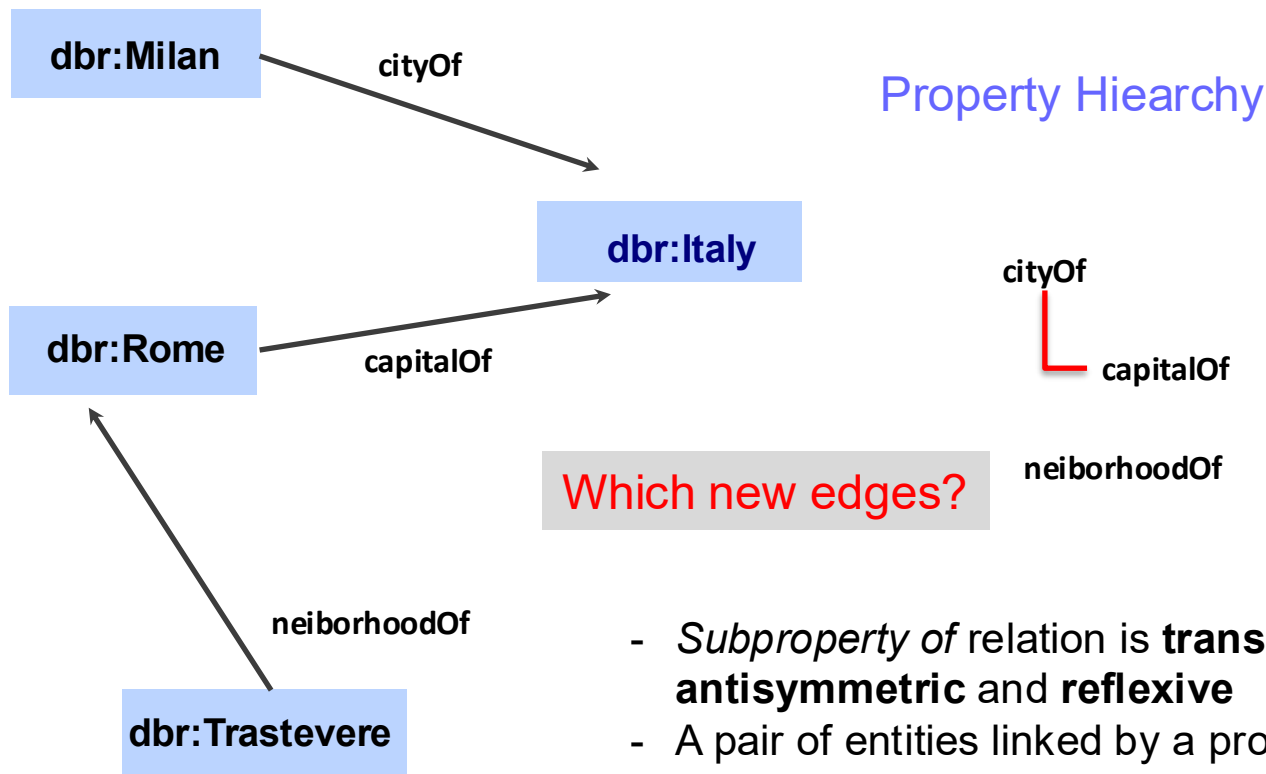


if	then
<code>x rdfs:subClassOf y .</code> <code>a rdf:type x .</code>	<code>a rdf:type y .</code>
<code>x rdfs:subClassOf y .</code> <code>y rdfs:subClassOf z .</code>	<code>x rdfs:subClassOf z .</code>
<code>x p y .</code> <code>p rdfs:subPropertyOf q .</code>	<code>x q y .</code>
<code>p rdfs:subPropertyOf q .</code> <code>q rdfs:subPropertyOf r .</code>	<code>p rdfs:subPropertyOf r .</code>
<code>x p y .</code> <code>p rdfs:domain z .</code>	<code>x rdf:type z .</code>
<code>x p u .</code> <code>p rdfs:range z .</code>	<code>u rdf:type z .</code>

+ rdfs6
(reflexivity)

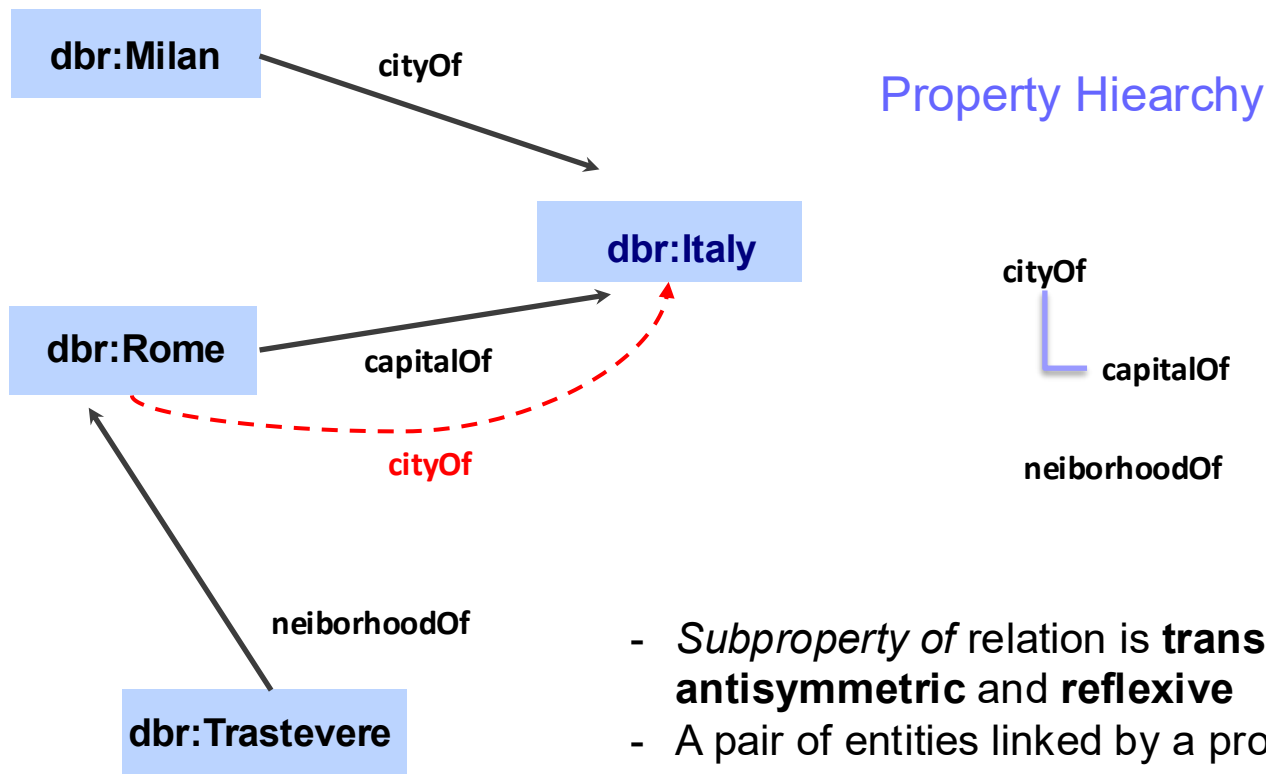
Read out more in RDF Semantics <http://www.w3.org/TR/rdf-mt/>

Minimal Reasoning with RDFS



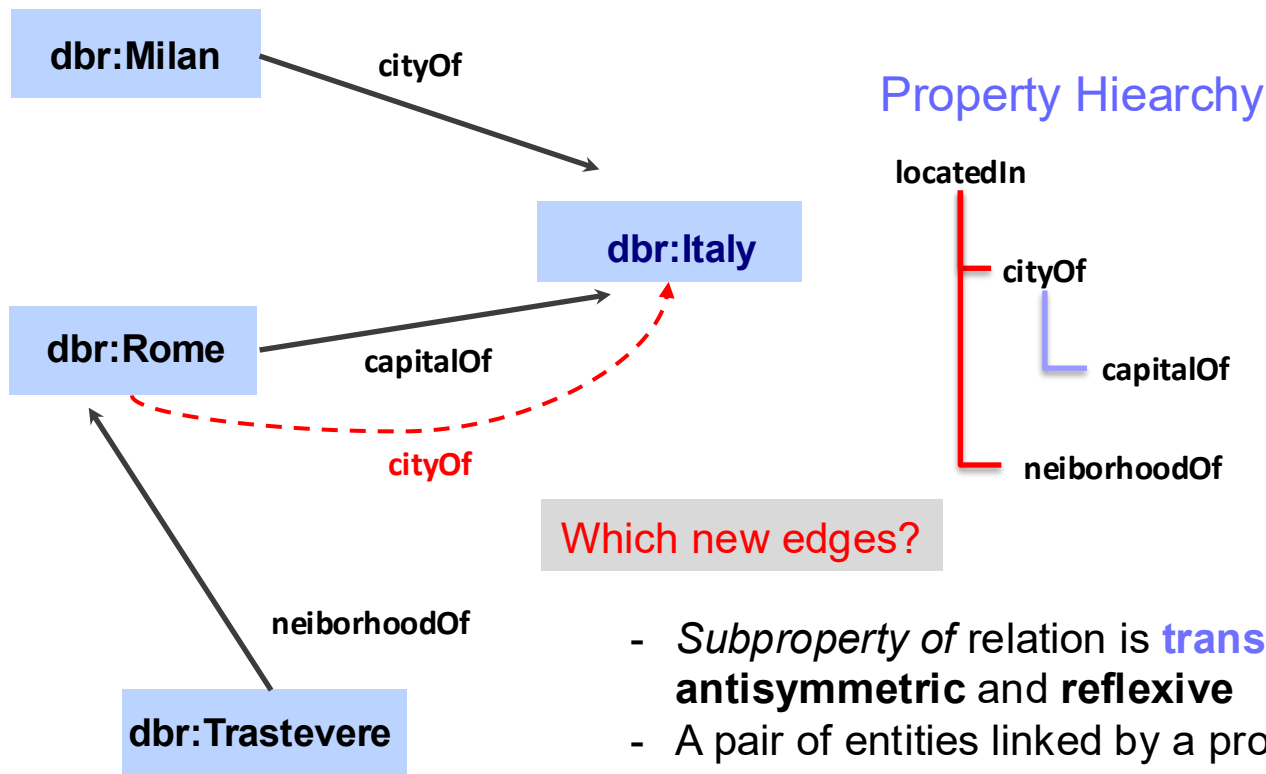
- Subproperty of relation is **transitive**, **antisymmetric** and **reflexive**
- A pair of entities linked by a property are also linked by all its super properties

Minimal Reasoning with RDFS



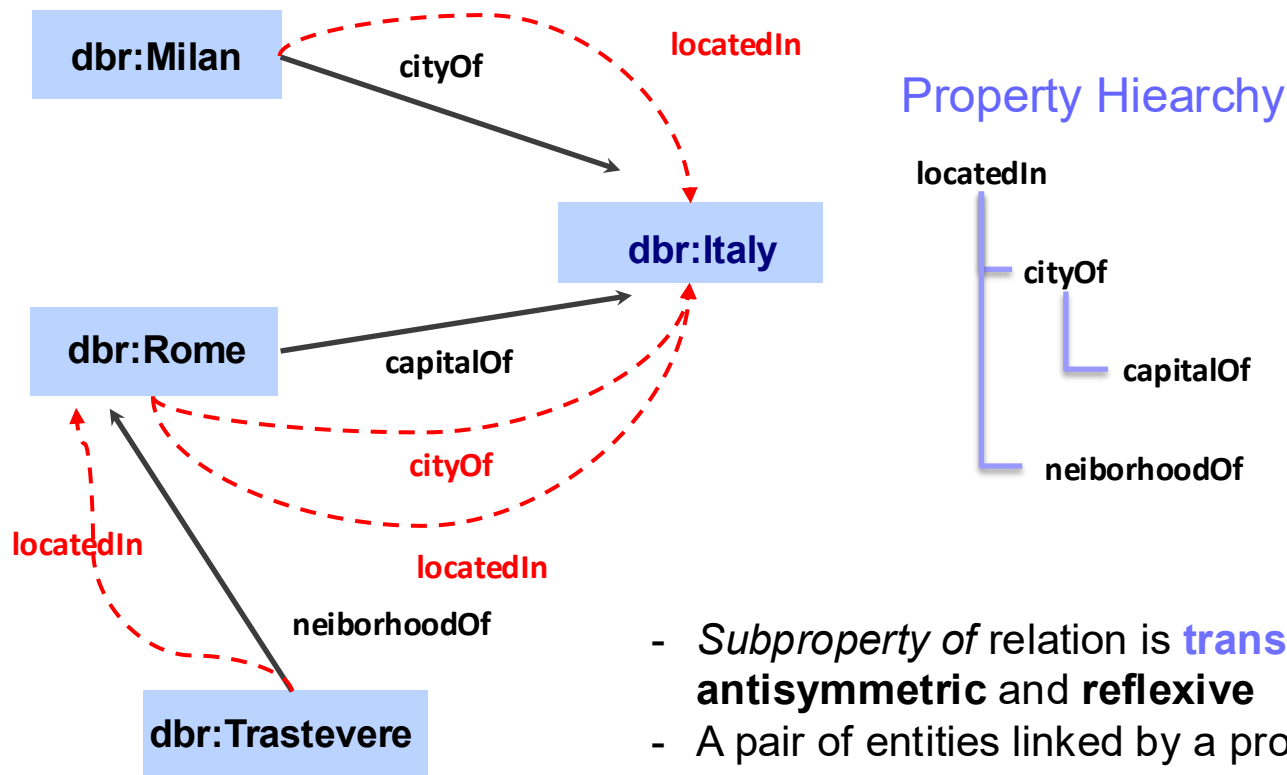
- Subproperty of relation is **transitive**, **antisymmetric** and **reflexive**
- A pair of entities linked by a property are also linked by all its super properties

Minimal Reasoning with RDFS



- Subproperty of relation is **transitive**, **antisymmetric** and **reflexive**
- A pair of entities linked by a property are also linked by all its super properties

Minimal Reasoning with RDFS



- Subproperty of relation is **transitive**, **antisymmetric** and **reflexive**
- A pair of entities linked by a property are also linked by all its super properties

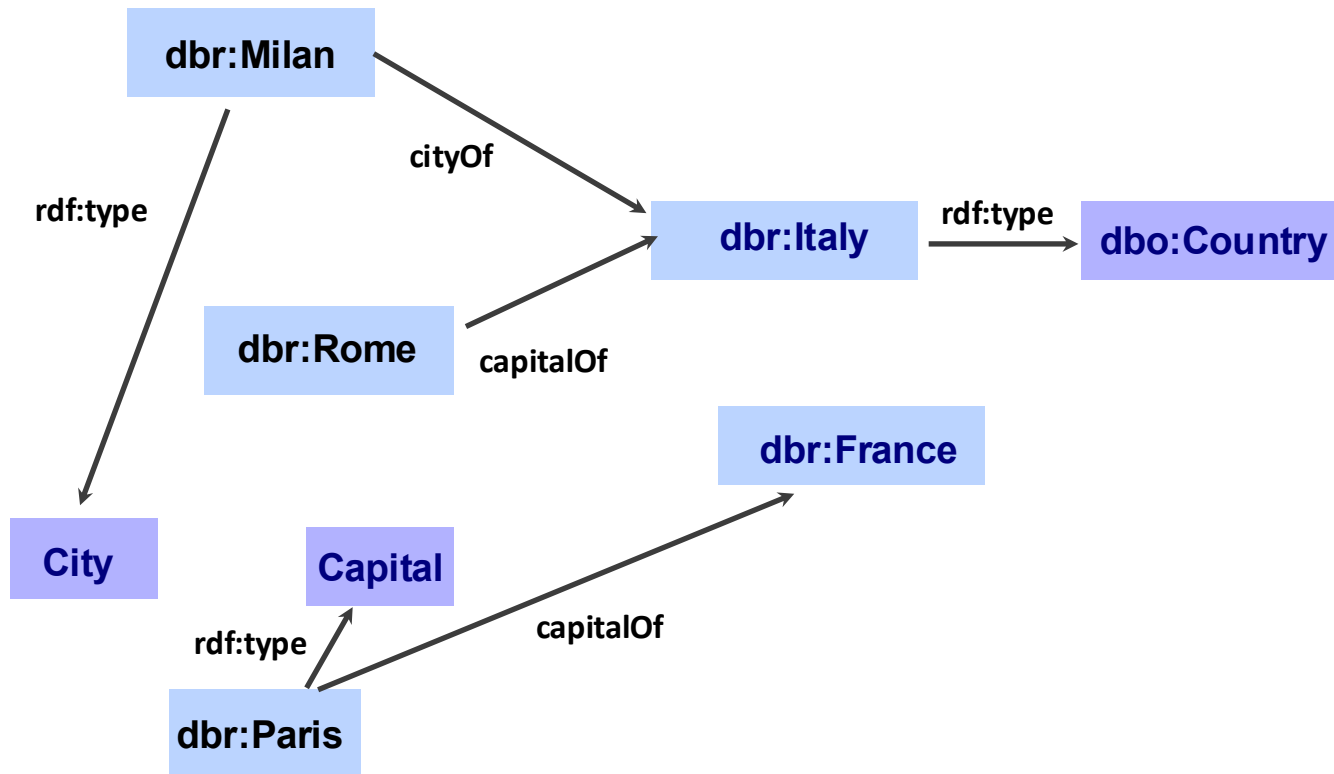


RDFS semantics (core part of it)

if	then
<pre>x rdfs:subClassOf y . a rdf:type x .</pre>	<pre>a rdf:type y .</pre>
<pre>x rdfs:subClassOf y . y rdfs:subClassOf z .</pre>	<pre>x rdfs:subClassOf z .</pre>
<pre>x p y . p rdfs:subPropertyOf q .</pre>	<pre>x q y .</pre>
<pre>p rdfs:subPropertyOf q . q rdfs:subPropertyOf r .</pre>	<pre>p rdfs:subPropertyOf r .</pre>
<pre>x p y . p rdfs:domain z .</pre>	<pre>x rdf:type z .</pre>
<pre>x p u . p rdfs:range z .</pre>	<pre>u rdf:type z .</pre>

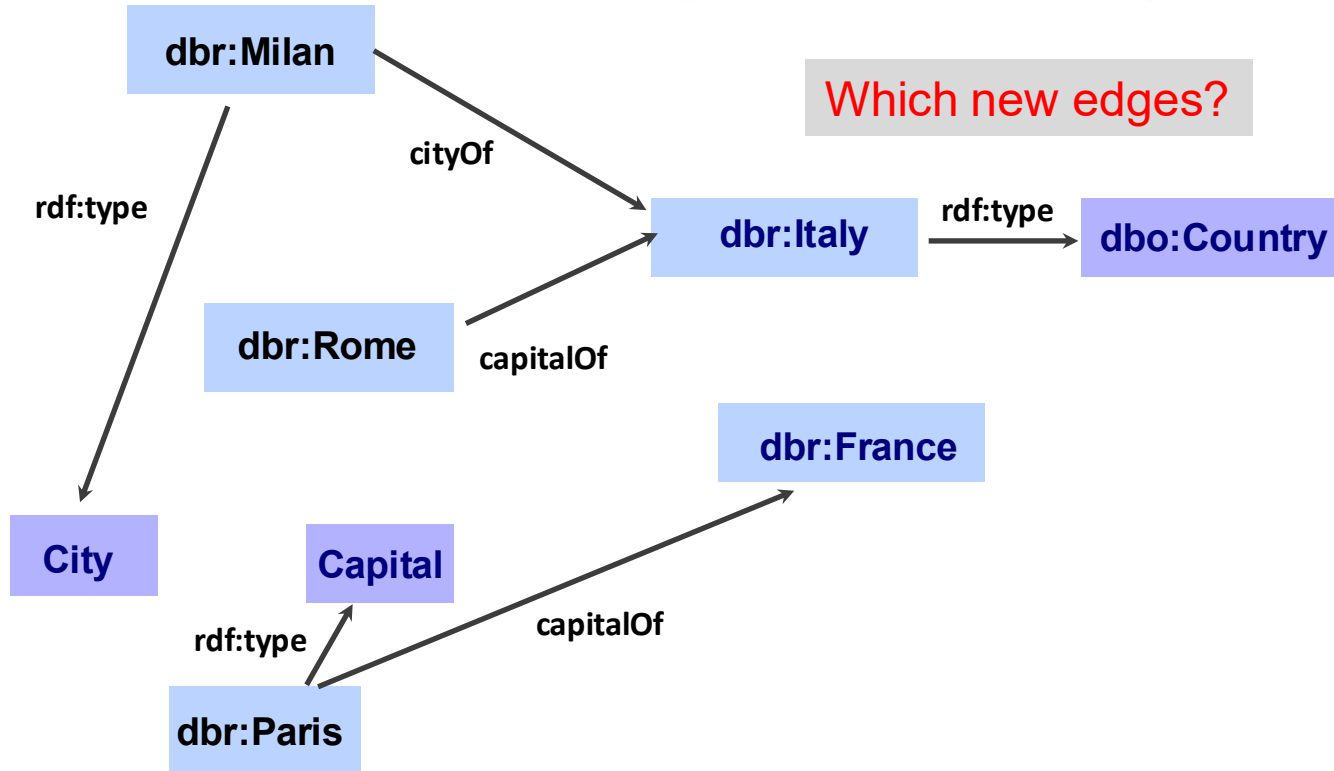
Read out more in RDF Semantics <http://www.w3.org/TR/rdf-mt/>

Minimal Reasoning with RDFS



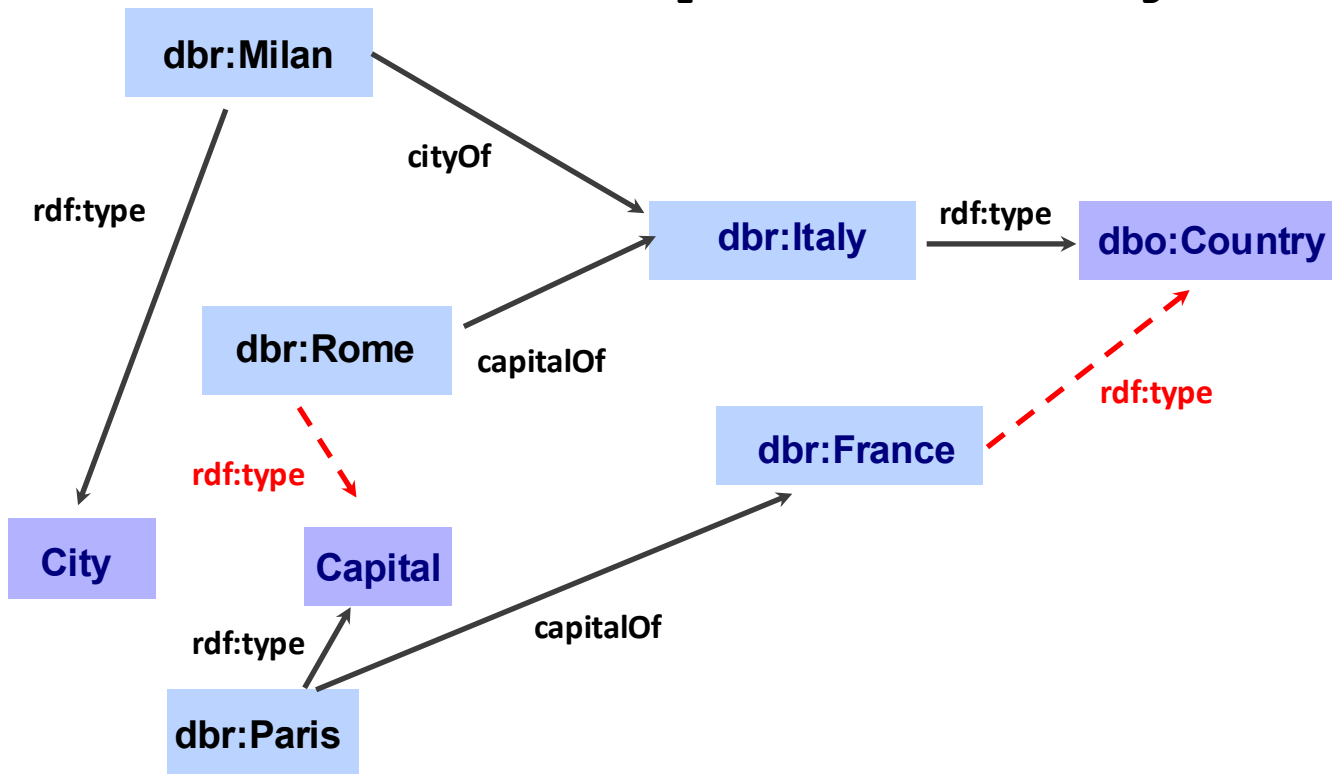
Minimal Reasoning with RDFS

```
capitalOf rdfs:domain dbo:Capital .  
capitalOf rdfs:range  dbo:Country  .
```



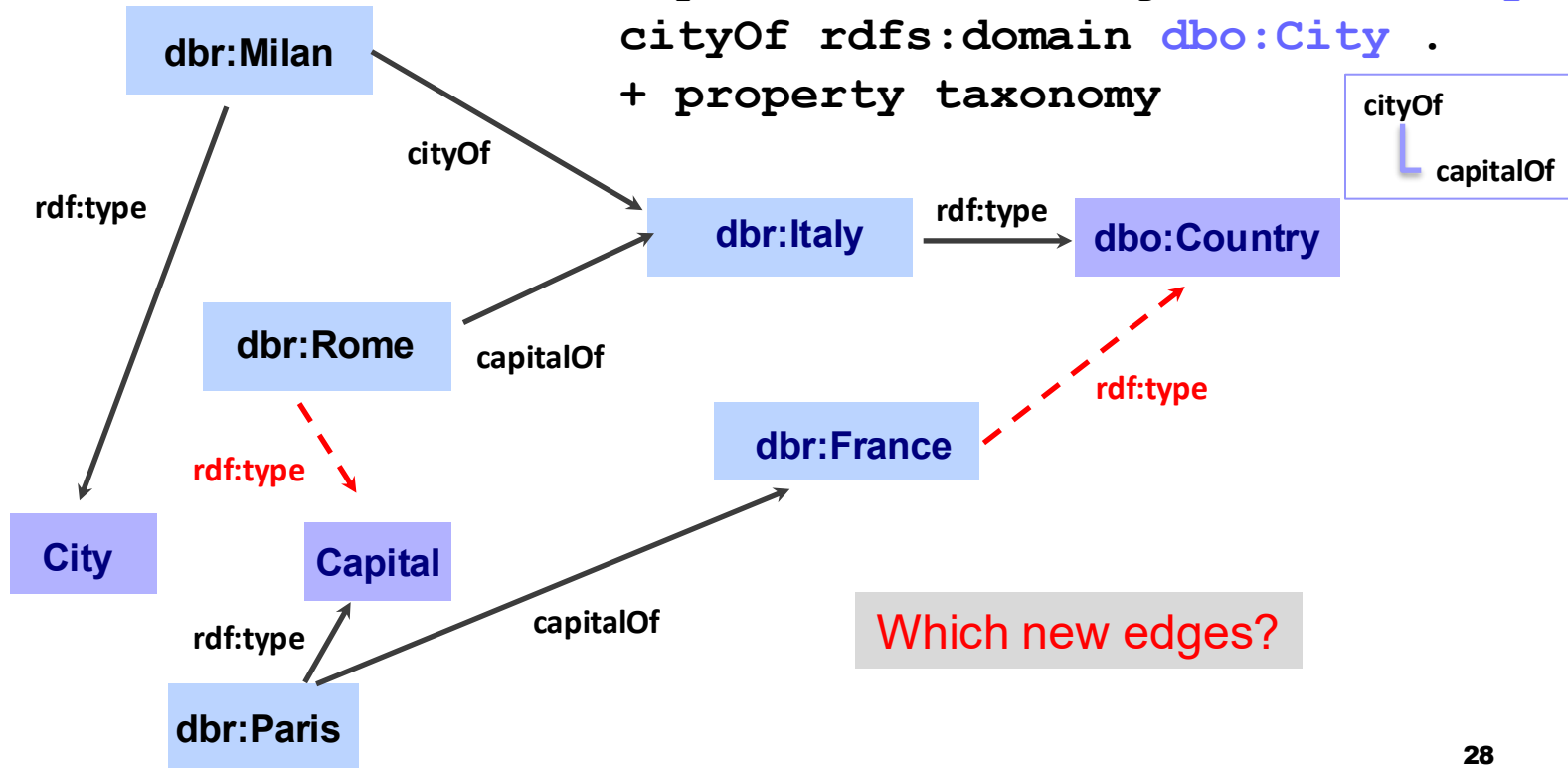
Minimal Reasoning with RDFS

```
capitalOf rdfs:domain dbo:Capital .  
capitalOf rdfs:range  dbo:Country .
```



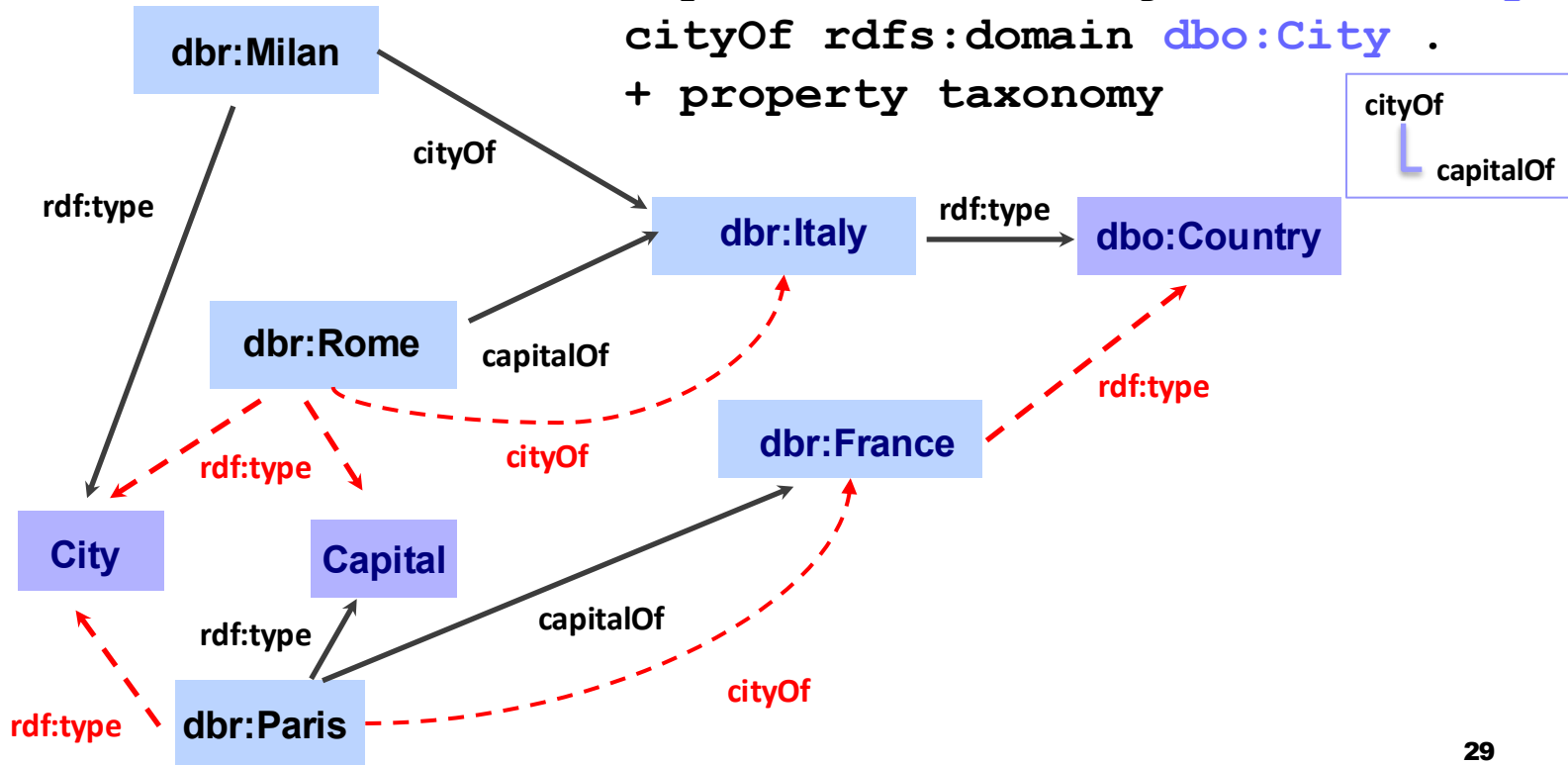
Minimal Reasoning with RDFS

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country .
cityOf rdfs:domain  dbo:City .
+ property taxonomy
```



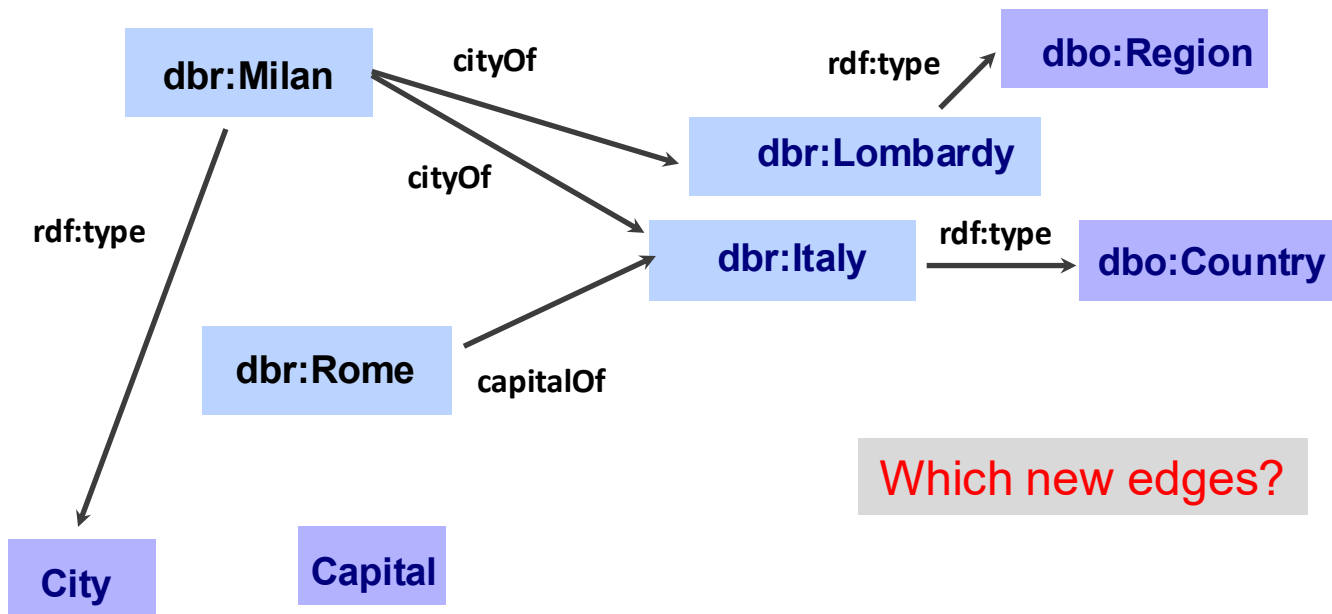
Not-so-minimal Reasoning with RDFS

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country .
cityOf rdfs:domain  dbo:City .
+ property taxonomy
```



Not-so-minimal Reasoning with RDFS

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country  .
cityOf rdfs:range  dbo:Country  .
+ property taxonomy
```

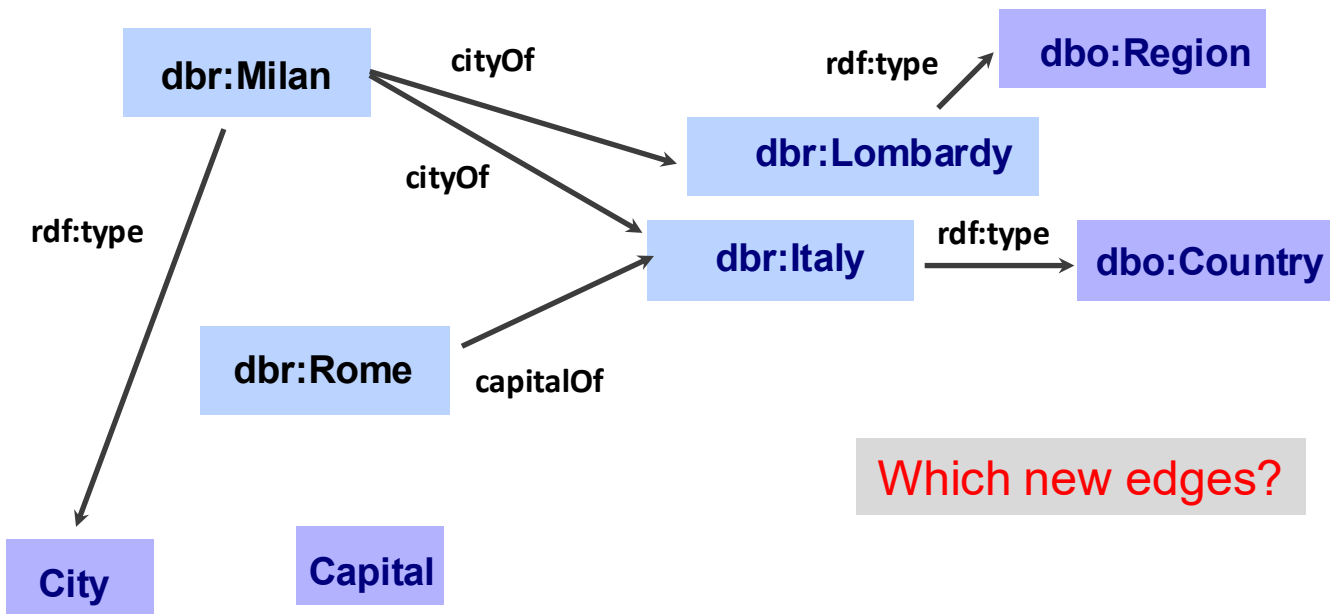


Not-so-minimal Reasoning with RDFS

Attention to unintended and
undesired consequences!

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country .
cityOf rdfs:range  dbo:Country .
+ property taxonomy
```

cityOf
└ capitalOf

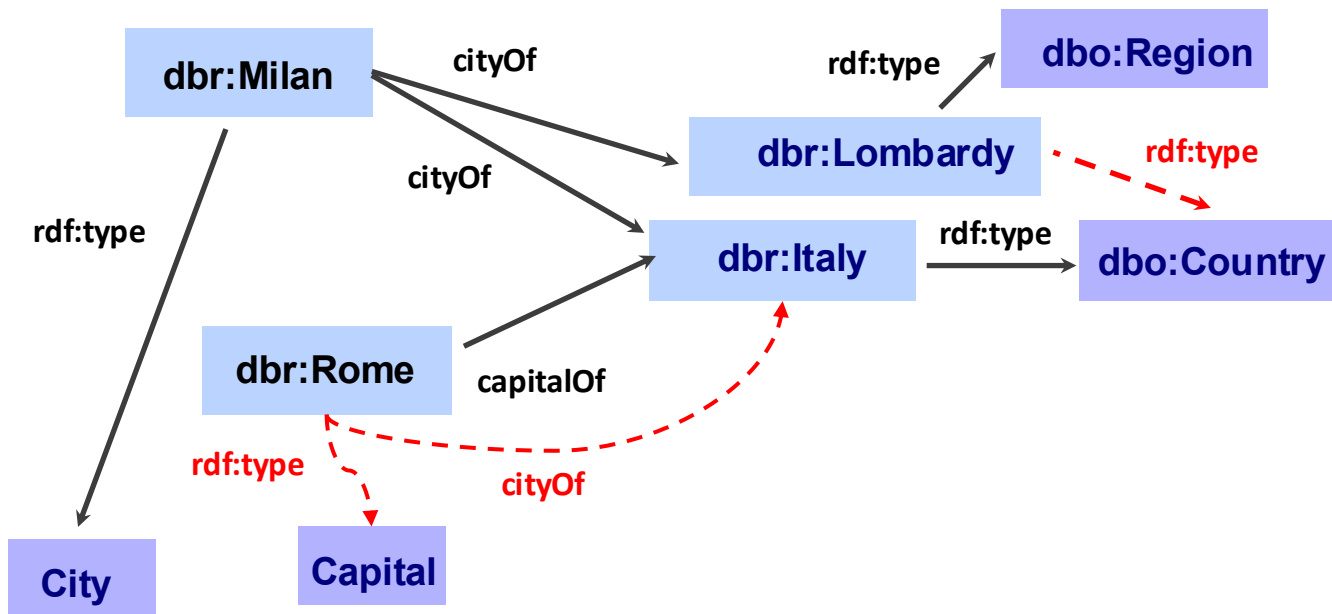


Not-so-minimal Reasoning with RDFS

Attention to unintended and
undesired consequences!

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country .
cityOf rdfs:range  dbo:Country .
+ property taxonomy
```

cityOf
└ capitalOf

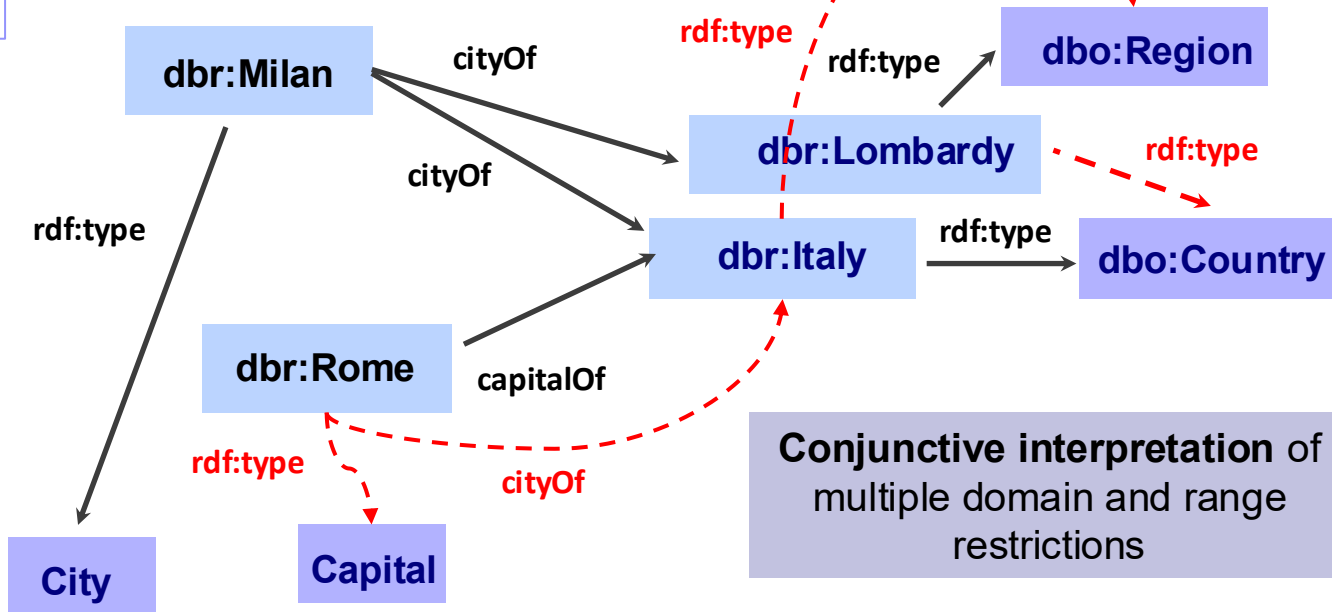


Not-so-minimal Reasoning with RDFS

+ property taxonomy

Attention to unintended and undesired consequences!

```
capitalOf rdfs:domain dbo:Capital .
capitalOf rdfs:range  dbo:Country .
cityOf    rdfs:range  dbo:Country .
cityOf    rdfs:range  dbo:Region .
```



Not-so-minimal Reasoning with RDFS

Attention to unintended and undesired consequences!

```
capitalOf rdfs:domain dbo:Capital .  
capitalOf rdfs:range  dbo:Country .  
cityOf    rdfs:range  dbo:Country .  
cityOf    rdfs:range  dbo:Region  .
```

RDFS not expressive enough for:
deriving inconsistencies
disjunctive interpretation of domain/range restrictions
weaker axioms ~ domain/range restrictions

Need OWL for this!

City

Capital

rdf:type

cityOf

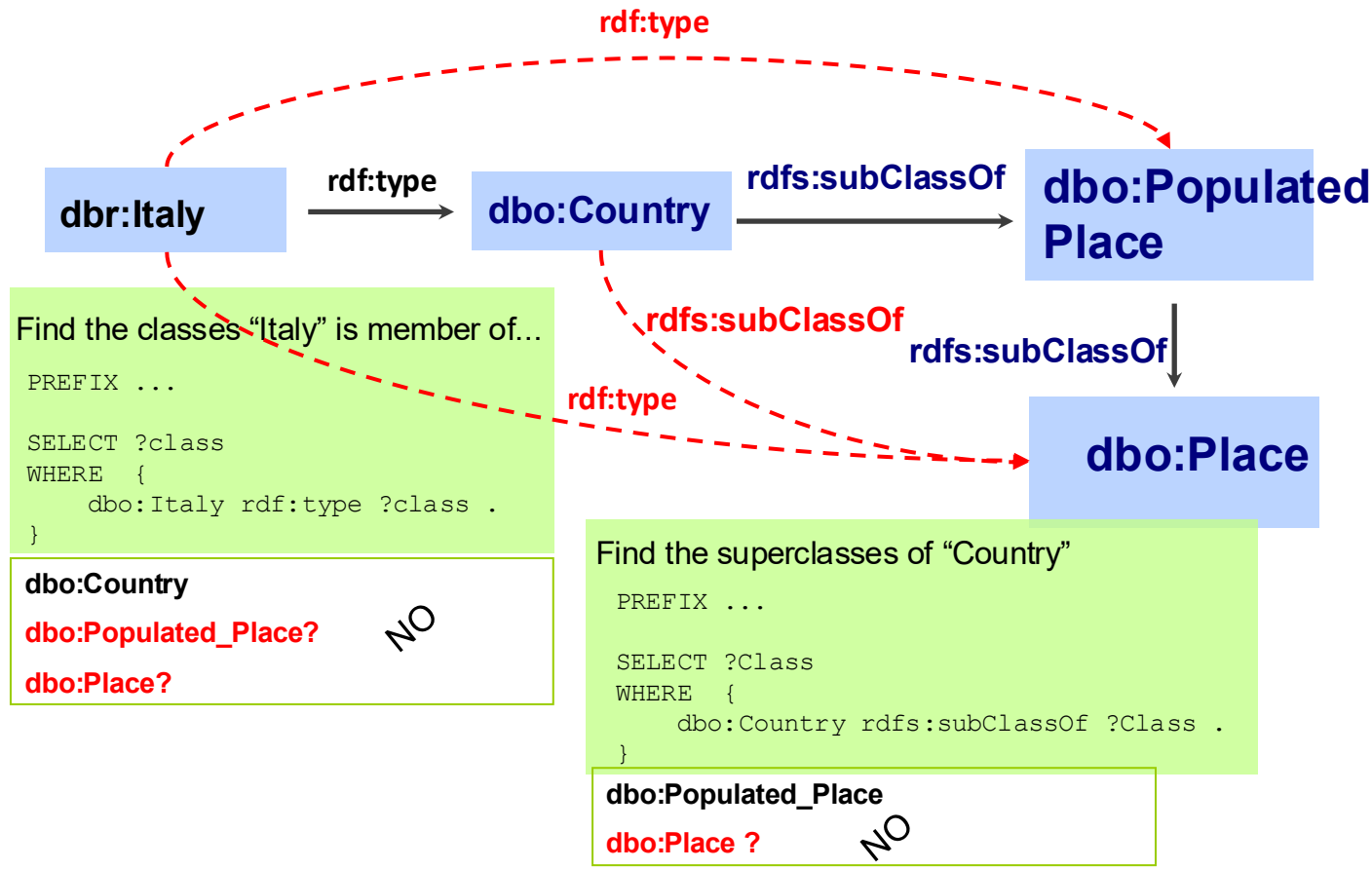
Conjunctive interpretation of
multiple domain and range
restrictions



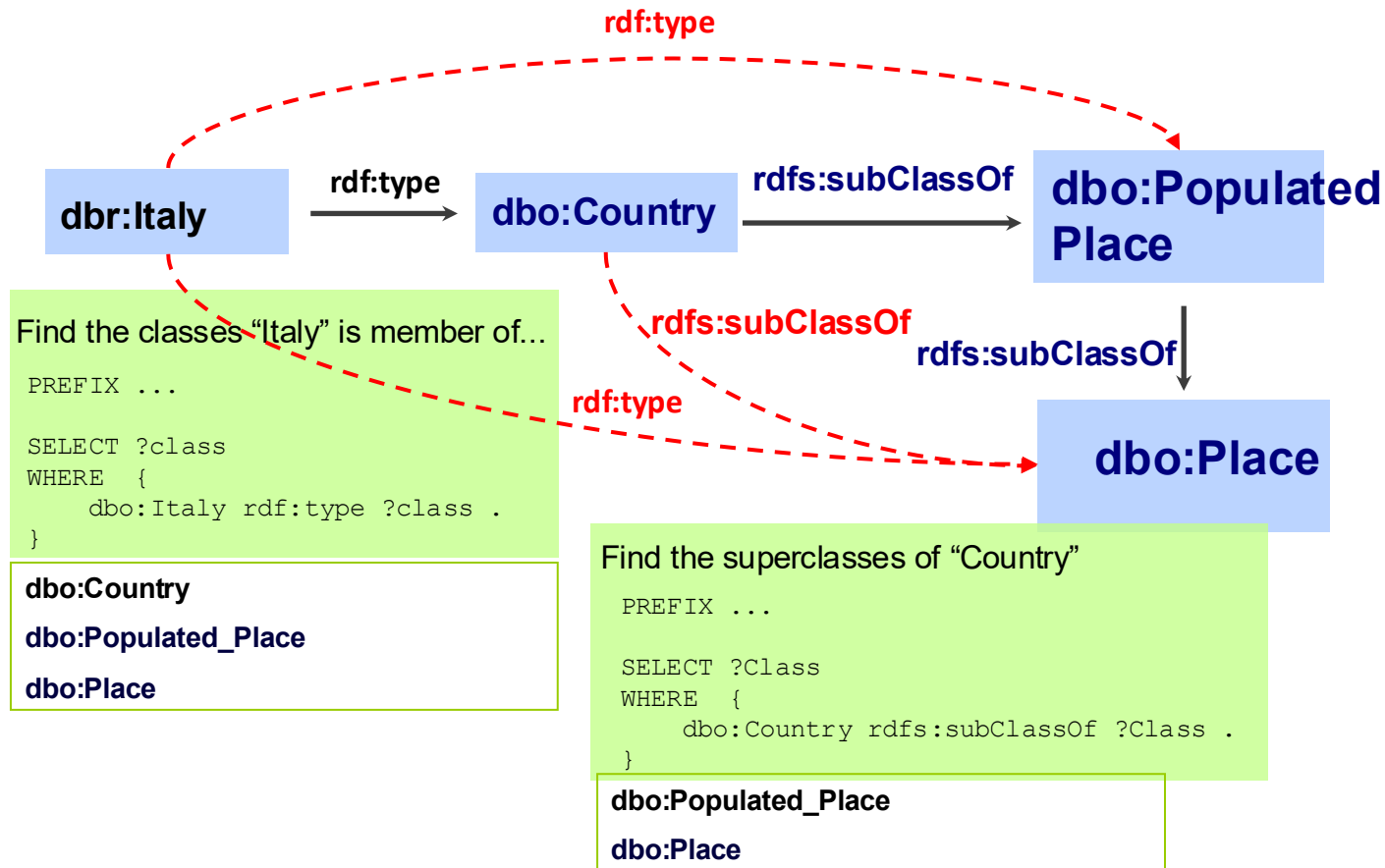
RDFS

2 – RDFS in Queries

(Pure) SPARQL Query Answering vs with Reasoning



(Pure) SPARQL Query Answering vs with Reasoning

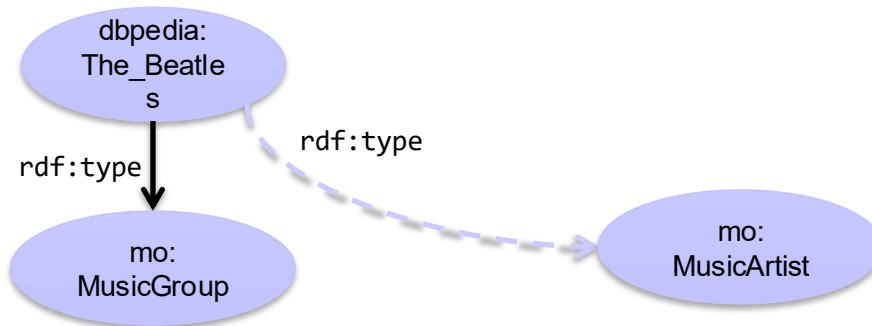


SPARQL 1.1: Entailment Regimes

- SPARQL 1.0 was defined only for simple entailment (pattern matching)
- SPARQL 1.1 is extended with entailment regimes other than simple entailment:
 - RDF entailment
 - **RDFS entailment**
 - D-Entailment
 - OWL RL entailment
 - OWL Full entailment
 - OWL 2 DL, EL, and QL entailment
 - RIF entailment

Source: <http://www.w3.org/TR/rdf-mt/#RDFSRules>

rdfs9 – rdfs:subClassOf



Schema:

```
mo:MusicGroup rdfs:subClassOf
mo:MusicArtist .
```

Result set:

?x

Query:

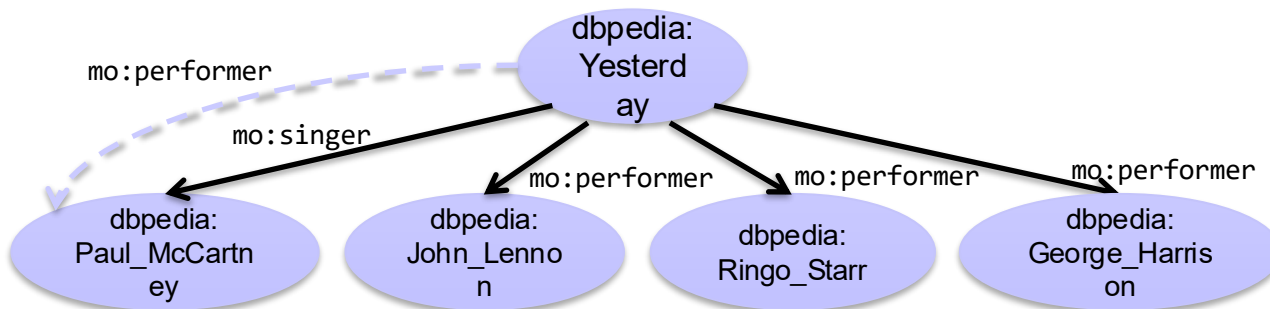
```
SELECT ?x WHERE {
  ?x a mo:MusicArtist.}
```

Result set with inference:

?x

dbpedia:The_Beatles ...

rdfs7 – rdfs:subPropertyOf



Schema:

```
mo:singer rdfs:subPropertyOf
mo:performer .
```

Query:

```
SELECT ?x WHERE {
  dbpedia:Yesterday mo:performer ?x.}
```

Result set:

?x

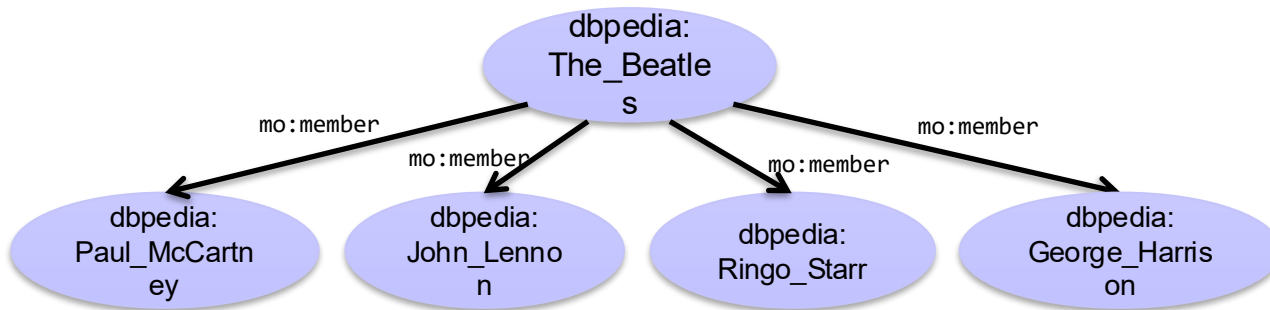
dbpedia:John_Lennon
dbpedia:Ringo_Starr
dbpedia:George_Harrison

Result set with inference:

?x

dbpedia:John_Lennon
dbpedia:Ringo_Starr
dbpedia:George_Harrison
dbpedia:Paul_McCartney

rdfs2 – rdfs:domain



Schema:

```
mo:member rdfs:domain
           mo:MusicGroup .
```

Query:

```
SELECT ?x WHERE {
  ?x a mo:MusicGroup.}
```

Result set:

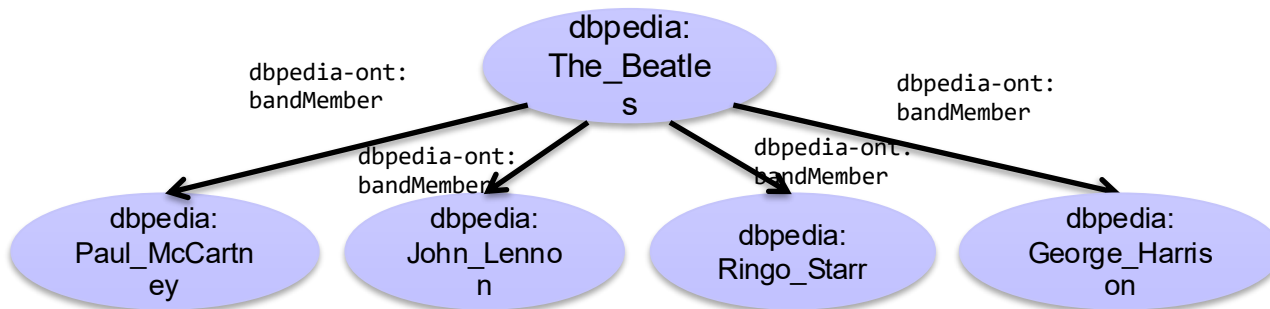
?x

Result set with inference:

?x

dbpedia:The_Beatles ...

rdfs3 – rdfs:range



Schema:

```
mo:member rdfs:range
           foaf:Agent .
```

Query:

```
SELECT ?x WHERE {
  ?x a foaf:Agent.}
```

Result set:

?x

Result set with inference:

?x

dbpedia:Paul_McCartney

dbpedia:John_Lennon

dbpedia:Ringo_Starr

dbpedia:George_Harrison ...

Approaches to Combine Querying & Inference (~ all languages)

- Response time and querying with inference
 - Querying with entailment regimes different from RDF requires reasoning on-the-fly: overhead on the response time (efficient for RDFs but less efficient for more expressive languages)
- Two strategies for reasoning with RDF data:
 - **Materialization**: compute all the inferred triples before publishing the data + store the inferred triples in the triple store + pure query answering; e.g., DBpedia
 - +) better ? (esp. on large data)
 - -) more difficult to deal with?
 - **Inference at runtime**: resolve queries by considering inferred triples
 - -) worse ? (esp. on large data)
 - +) easily to deal with ?

Approaches to Combine Querying & Inference (~ all languages)

- Response time and querying with inference
 - Querying with entailment regimes different from RDF requires reasoning on-the-fly: overhead on the response time (efficient for RDFs but less efficient for more expressive languages)
- Two strategies for reasoning with RDF data:
 - **Materialization**: compute all the inferred triples before publishing the data + store the inferred triples in the triple store + pure query answering; e.g., DBpedia
 - +) better **response time** (esp. on large data)
 - -) more difficult to deal with **changes in the data** (inferred triples must be computed again)
 - **Inference at runtime**: resolve queries by considering inferred triples
 - -) worse **response time** (esp. on large data)
 - +) easily to deal with **changes in the data** (don't need to compute inferences again)

Wrap-up

- RDFS: basic inferences for RDF
- A simple ontology language
 - Key and useful inferences (“little semantics goes a long way”)
 - No contradiction
 - Several limitations
- Example of rule-based semantics
 - Rules specify the inferences to be computed
 - Rules applied until a fix point is reached

Most likely in the next lesson

IF WE HAVE TIME

Data Schemas: RDBs vs RDF

■ RDF data: semi-structured data

- Some structure is given by the data model (triples)
- Data and schema are **loosly coupled**
 - Data must use URIs for properties and are expected to use URIs for classes → **vocabularies**
 - In principle... “data-first”: data can be inserted without any schema definition, what is used is the vocabulary (not a good practice: remember, one of our goal is use terms with shared meaning)
 - Non-prescriptive “schemas”: data are not required to be compliant with the schema to be inserted
 - Schema as “social agreement”: community-based definitions of recommended terms and usage
 - Deductive schema: the schema defines the meaning of the terms in the vocabulary in terms of inferences (e.g., triples added to the KG, inconsistencies)

Often referred
to as
“ontologies”

```
<dbp:Empty_Sky, dbo:artist, dbp:Elton_John>
<dbp:Sails, dbo:musicalArtist, dbp:Elton_John>
<dbp:Sails, dbo:album, dbp:Empty_Sky>
<dbp:Elton_John, dbo:instrument, dbp:Electric_Piano>
<dbp:Elton_John, rdf:type, dbp:Musical_Artist>
<dbp:Sails, rdf:type, dbo:Single>
<dbp:Empty_Sky, rdf:type, dbo:Album>
<dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
<dbo:Artist, rdfs:subClassOf, dbo:Person>
...
dbp: for http://dbpedia.org/resource/
      e.g.: http://dbpedia.org/resource/Elton\_John
dbo: for http://dbpedia.org/ontology/
rdfs for https://www.w3.org/2000/01/rdf-schema/
rdf for http://www.w3.org/1999/02/22-rdf-syntax-ns#
```

Logic-based
languages
RDFS/OWL

Query & Reasoning in Oracle 11g

Oracle 11g RDF/OWL Graph Data Management

- Oracle 11g is the leading commercial database with native RDF/OWL data management
- Scalable & secure platform for wide-range of semantic applications
- Readily scales to ultra-large repositories (+1 billion)
- Choice of SQL or SPARQL query
- Leverages Oracle Partitioning and Advanced Compression. RAC supported
- Growing ecosystem of 3rd party tools partners

Key Capabilities:

Load / Storage

- Native RDF graph data store
- Manages billions of triples
- Fast batch, bulk and incremental load

Query

- SQL: SEM_Match
- SPARQL: via Jena plug-in
- Ontology assisted query of RDBMS data

Reasoning

- Forward chaining model
- RDFS++ OWL, OWL Prime
- User defined rule base



ORACLE

More about Oracle Graph Techs

■ Oracle 19c

- <https://www.oracle.com/a/tech/docs/introduction-to-rdf-graph-for-oracle-database19c.pdf>

■ Oracle 21c

- <https://www.oracle.com/it/database/graph/>

■ 17 Use Cases for Graph Databases and Graph Analytics

- <https://www.oracle.com/big-data/what-is-graph-database/graph-databases-and-graph-analytics-ebook/>

Exercises

- Exercises from the KRR2 course
 - Exercises
 - Solutions